Logistic Regression

Comprehensive Cheatsheet

Machine Intelligence - Andrés Masegosa - Fall 2025

Contents

1 From Regression to Classification **2**

1.1 Two Types of Machine Learning Tasks 2

1.2 Binary Classification 2

2 The Sigmoid Function **3**

2.1 Definition and Properties 3

3 Logistic Regression Model **5**

3.1 The Model 5

4 Decision Boundaries **6**

4.1 Understanding Decision Boundaries 6

4.2 Linear Decision Boundaries 7

4.3 Non-Linear Decision Boundaries 8

5 Cost Function for Logistic Regression **9**

5.1 Why Not Use Squared Error? 9

5.2 Logistic Regression Cost Function 9

5.3 Total Cost Function 10

6 Gradient Descent for Logistic Regression **11**

6.1 The Algorithm 11

7 Overfitting in Classification **14**

7.1 Understanding Overfitting 14

8 Addressing Overfitting **16**

8.1 Three Main Strategies 16

9 Regularization **18**

9.1 The Concept 18

9.2 Regularization Term 19
9.3 Regularized Cost Function 21

10 Regularized Gradient Descent **22**
10.1 Updated Gradient Formulas 22
10.2 Gradient Descent Algorithm 23

11 Key Formulas Summary **26**

12 Summary and Key Concepts **28**

1 From Regression to Classification

1.1 Two Types of Machine Learning Tasks

Regression vs Classification

Regression Tasks:

- **Output**: Continuous numerical value (double/float)
- **Examples**:
  - Predict house price: \$250,000
  - Predict temperature: 23.5°C
  - Predict stock price: \$142.75
- **Model**: $f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$ (can output any real number)

Classification Tasks:

- **Output**: Discrete category or class (boolean/label)
- **Examples**:
  - Is this email spam? (Yes/No)
  - Is this transaction fraudulent? (Yes/No)
  - Is this tumor malignant? (Yes/No)
- **Model**: $f_{\vec{w},b}(\vec{x})$ outputs probability between 0 and 1

Key Difference: Regression predicts "how much", classification predicts "which category"

1.2 Binary Classification

Binary Classification Setup

Goal: Classify input into one of TWO classes
Standard Notation:

- **Output labels**: $y \in \{0, 1\}$
  - $y = 1$: Positive class (e.g., "yes", "true", "malignant")
  - $y = 0$: Negative class (e.g., "no", "false", "benign")
- **Training data**: $\{(\vec{x}^{(1)}, y^{(1)}), (\vec{x}^{(2)}, y^{(2)}), \ldots, (\vec{x}^{(m)}, y^{(m)})\}$
- Each $y^{(i)}$ is either 0 or 1

Examples:

Question	Answer y	Interpretation
Is this email spam?	1 or 0	1 = spam, 0 = not spam
Is the transaction fraudulent?	1 or 0	1 = fraud, 0 = legitimate
Is the tumor malignant?	1 or 0	1 = malignant, 0 = benign

Why Not Use Linear Regression?

Problem: Linear regression outputs can be any value: $f(\vec{x}) \in (-\infty, \infty)$
For classification, we need:

- Output should be between 0 and 1 (representing probability)
- We can then threshold: if $f(\vec{x}) \geq 0.5$, predict $y = 1$, else predict $y = 0$

Issues with Linear Regression for Classification:

- Can output values < 0 or > 1 (not valid probabilities)
- Sensitive to outliers (one extreme point can shift the entire line)
- Doesn't capture the natural S-shaped decision boundary

Solution: Use logistic regression with sigmoid function!

2 The Sigmoid Function

2.1 Definition and Properties

Sigmoid Function (Logistic Function)

Formula:

$$g(z) = \frac{1}{1 + e^{-z}}$$

Key Properties:

- **Range**: $g(z) \in (0, 1)$ for all $z \in \mathbb{R}$
- **Output interpretation**: Probability that $y = 1$
- **S-shaped curve**: Smooth transition from 0 to 1
- **Midpoint**: $g(0) = 0.5$
- **Asymptotes**:
  - As $z \to \infty$: $g(z) \to 1$
  - As $z \to -\infty$: $g(z) \to 0$
- **Symmetry**: $g(-z) = 1 - g(z)$

Understanding the Sigmoid

Intuition: The sigmoid "squashes" any real number into the range $(0, 1)$
Behavior:

Input z	Output $g(z)$	Interpretation
$z = -\infty$	≈ 0	Very confident $y = 0$
$z = -5$	≈ 0.007	Strong evidence for $y = 0$
$z = 0$	0.5	Uncertain (50-50)
$z = 5$	≈ 0.993	Strong evidence for $y = 1$
$z = \infty$	≈ 1	Very confident $y = 1$

Plain English:

- Large positive $z \to$ output close to 1 $\to$ predict $y = 1$
- Large negative $z \to$ output close to 0 $\to$ predict $y = 0$
- z near 0 $\to$ output near 0.5 $\to$ uncertain prediction

Computing Sigmoid Values

Example calculations:
For $z = 2$:

$$g(2) = \frac{1}{1 + e^{-2}} = \frac{1}{1 + 0.135} = \frac{1}{1.135} \approx 0.88$$

For $z = -3$:

$$g(-3) = \frac{1}{1 + e^{3}} = \frac{1}{1 + 20.09} = \frac{1}{21.09} \approx 0.047$$

In Python:

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# Works for scalars and arrays
sigmoid(0)      # Returns 0.5
sigmoid([0, 2, -3])  # Returns [0.5, 0.88, 0.047]
```

3 Logistic Regression Model

3.1 The Model

Logistic Regression Model

Formula:

$$f_{\vec{w},b}(\vec{x}) = g(\vec{w} \cdot \vec{x} + b) = \frac{1}{1 + e^{-(\vec{w} \cdot \vec{x} + b)}}$$

Components:

- $\vec{x} = [x_1, x_2, \ldots, x_n]$: Input features
- $\vec{w} = [w_1, w_2, \ldots, w_n]$: Weights (parameters)
- b: Bias term (parameter)
- $z = \vec{w} \cdot \vec{x} + b$: Linear combination (same as linear regression!)
- $g(z)$: Sigmoid function applied to z

Output Interpretation:

$$f_{\vec{w},b}(\vec{x}) = P(y = 1 \mid \vec{x}; \vec{w}, b)$$

The output is the probability that $y = 1$ given input $\vec{x}$ and parameters $\vec{w}, b$

How Logistic Regression Works

Step-by-Step Process:
Step 1: Compute linear combination (just like linear regression)

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$$

Step 2: Apply sigmoid function to get probability

$$f_{\vec{w},b}(\vec{x}) = g(z) = \frac{1}{1 + e^{-z}}$$

Step 3: Make prediction

- If $f_{\vec{w},b}(\vec{x}) \geq 0.5$: Predict $\hat{y} = 1$ (positive class)
- If $f_{\vec{w},b}(\vec{x}) < 0.5$: Predict $\hat{y} = 0$ (negative class)

Key Insight: Logistic regression = Linear regression + Sigmoid function

- Linear part: $z = \vec{w} \cdot \vec{x} + b$ (same as linear regression)
- Nonlinear part: $g(z)$ (sigmoid squashes to $[0, 1]$)

Tumor Classification Example

Task: Predict if tumor is malignant based on size
Suppose we learned:

- $w_1 = 0.5$ (weight for tumor size)
- $b = -5$ (bias)

Prediction for tumor size = 4 cm:
Step 1: Compute z:

$$z = w_1 x_1 + b = 0.5(4) + (-5) = 2 - 5 = -3$$

Step 2: Apply sigmoid:

$$f_{w,b}(x) = g(-3) = \frac{1}{1 + e^3} \approx 0.047$$

Step 3: Make decision:

- $0.047 < 0.5 \rightarrow$ Predict $\hat{y} = 0$ (benign)
- Interpretation: Only 4.7% probability of being malignant

Prediction for tumor size = 12 cm:
Step 1: $z = 0.5(12) - 5 = 6 - 5 = 1$
Step 2: $f_{w,b}(x) = g(1) \approx 0.73$
Step 3: $0.73 \geq 0.5 \rightarrow$ Predict $\hat{y} = 1$ (malignant, 73% probability)

4 Decision Boundaries

4.1 Understanding Decision Boundaries

Decision Boundary

Definition: The boundary that separates regions where we predict $y = 1$ from regions where we predict $y = 0$
Mathematical Definition: The decision boundary occurs where:

$$f_{\vec{w},b}(\vec{x}) = 0.5$$

Since $g(z) = 0.5$ when $z = 0$, the decision boundary is:

$$\vec{w} \cdot \vec{x} + b = 0$$

Prediction Rules:

- If $\vec{w} \cdot \vec{x} + b \geq 0$: Predict $\hat{y} = 1$
- If $\vec{w} \cdot \vec{x} + b < 0$: Predict $\hat{y} = 0$

Plain English: The decision boundary is where the model is exactly 50-50 uncertain. On one side, it predicts class 1; on the other side, class 0.

4.2 Linear Decision Boundaries

Linear Decision Boundary - 2 Features

Model:

$$f_{\vec{w},b}(\vec{x}) = g(w_1x_1 + w_2x_2 + b)$$

Decision Boundary:

$$w_1x_1 + w_2x_2 + b = 0$$

This is a straight line in 2D space!
Example: Suppose $w_1 = 1$, $w_2 = 1$, $b = -3$
Decision boundary:

$$x_1 + x_2 - 3 = 0 \quad \Rightarrow \quad x_2 = 3 - x_1$$

Interpretation:

- Points above the line ($x_1 + x_2 > 3$): Predict $y = 1$
- Points below the line ($x_1 + x_2 < 3$): Predict $y = 0$
- Points on the line: $f(\vec{x}) = 0.5$ (uncertain)

Visualizing Decision Boundaries

For 2 features (x_1, x_2):

- Feature space is 2D plane
- Decision boundary is a line
- Line divides plane into two regions
- One region $\rightarrow$ predict $y = 1$ (e.g., red X's)
- Other region $\rightarrow$ predict $y = 0$ (e.g., blue O's)

For n features:

- Feature space is n-dimensional
- Decision boundary is an $(n-1)$-dimensional hyperplane
- Hyperplane divides space into two half-spaces

Key Point: The decision boundary depends only on the weights $\vec{w}$ and bias b, not on the training data directly!

4.3 Non-Linear Decision Boundaries

Need for Non-Linear Boundaries

Problem: Many real-world problems need curved decision boundaries
Examples:

- Circular boundary: Points inside circle are class 1, outside are class 0
- Complex shapes: Medical diagnosis might need complex boundaries

Solution: Use polynomial features! (Same idea as polynomial regression)

Circular Decision Boundary

Goal: Classify points inside a circle as $y = 1$, outside as $y = 0$
Original features: x_1, x_2
Engineered features: Add polynomial terms

- $x_3 = x_1^2$
- $x_4 = x_2^2$

Model:

$$f_{\vec{w},b}(\vec{x}) = g(w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_2^2 + b)$$

Example parameters: $w_1 = 0$, $w_2 = 0$, $w_3 = 1$, $w_4 = 1$, $b = -1$
Decision boundary:

$$x_1^2 + x_2^2 - 1 = 0 \quad \Rightarrow \quad x_1^2 + x_2^2 = 1$$

This is a circle with radius 1!
Predictions:

- Inside circle ($x_1^2 + x_2^2 < 1$): Predict $y = 1$
- Outside circle ($x_1^2 + x_2^2 > 1$): Predict $y = 0$

Creating Complex Decision Boundaries

Strategy: Add polynomial and interaction features
Common transformations:

- Quadratic terms: x_1^2, x_2^2, x_1x_2
- Cubic terms: $x_1^3, x_2^3, x_1^2x_2, x_1x_2^2$
- Higher-degree polynomials for more complex shapes

Trade-off:

- **Benefit**: Can fit complex, non-linear decision boundaries
- **Risk**: Too many features can lead to overfitting
- **Solution**: Use regularization (covered later!)

Key Insight: Even with polynomial features, logistic regression is still "logistic" - we're just transforming the input space!

5 Cost Function for Logistic Regression

5.1 Why Not Use Squared Error?

Problem with Squared Error

Linear Regression Cost:

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})^2$$

Why this doesn't work for logistic regression:

- $f_{\vec{w},b}(\vec{x}) = \frac{1}{1+e^{-(\vec{w}\cdot\vec{x}+b)}}$ is non-linear (sigmoid)
- Squared error becomes non-convex (many local minima)
- Gradient descent might get stuck in local minimum
- No guarantee of finding the global minimum

Solution: Use a different cost function that is convex!

5.2 Logistic Regression Cost Function

Loss Function for Single Example

Loss for one training example $(\vec{x}^{(i)}, y^{(i)})$:

$$L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}) = \begin{cases} -\log(f_{\vec{w},b}(\vec{x}^{(i)})) & \text{if } y^{(i)} = 1 \\ -\log(1 - f_{\vec{w},b}(\vec{x}^{(i)})) & \text{if } y^{(i)} = 0 \end{cases}$$

Plain English:

- If actual label is $y = 1$: Loss is $-\log(f_{\vec{w},b}(\vec{x}))$
  - If model predicts $f = 1$ (confident correct): Loss $= -\log(1) = 0$ (perfect!)
  - If model predicts $f = 0.5$: Loss $= -\log(0.5) \approx 0.69$ (moderate penalty)
  - If model predicts $f \to 0$ (confident wrong): Loss $\to \infty$ (huge penalty!)
- If actual label is $y = 0$: Loss is $-\log(1 - f_{\vec{w},b}(\vec{x}))$
  - If model predicts $f = 0$ (confident correct): Loss $= -\log(1) = 0$ (perfect!)
  - If model predicts $f = 0.5$: Loss $= -\log(0.5) \approx 0.69$ (moderate penalty)
  - If model predicts $f \to 1$ (confident wrong): Loss $\to \infty$ (huge penalty!)

Why This Loss Function Works

Desirable Properties:

1. **Correct predictions have low loss**
   - $y = 1$ and f close to 1 $\rightarrow$ Loss near 0
   - $y = 0$ and f close to 0 $\rightarrow$ Loss near 0
2. **Wrong predictions have high loss**
   - $y = 1$ and f close to 0 $\rightarrow$ Loss very large
   - $y = 0$ and f close to 1 $\rightarrow$ Loss very large
3. **Smooth and differentiable**
   - Logarithm is smooth everywhere (except boundaries)
   - Can compute gradients for gradient descent
4. **Convex when combined with logistic model**
   - Total cost $J(\vec{w}, b)$ is convex
   - Gradient descent finds global minimum

Intuition: The loss function heavily penalizes confident wrong predictions, encouraging the model to output probabilities that match the true labels.

5.3 Total Cost Function

Logistic Regression Cost Function

Average loss over all training examples:

$$J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)})$$

Expanded form (using cases):

$$J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} \left[-y^{(i)} \log(f_{\vec{w},b}(\vec{x}^{(i)})) - (1 - y^{(i)}) \log(1 - f_{\vec{w},b}(\vec{x}^{(i)})) \right]$$

Compact form (works for both $y = 0$ and $y = 1$):

$$J(\vec{w}, b) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log(f_{\vec{w},b}(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f_{\vec{w},b}(\vec{x}^{(i)})) \right]$$

Why the compact form works:

- When $y^{(i)} = 1$: $(1 - y^{(i)}) = 0$, so second term vanishes $\rightarrow -\log(f_{\vec{w},b}(\vec{x}^{(i)}))$
- When $y^{(i)} = 0$: $y^{(i)} = 0$, so first term vanishes $\rightarrow -\log(1 - f_{\vec{w},b}(\vec{x}^{(i)}))$

Cost Function Summary

Goal: Find $\vec{w}$ and b that minimize $J(\vec{w}, b)$

$$\min_{\vec{w},b} J(\vec{w}, b) = \min_{\vec{w},b} \left\{ -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log(f_{\vec{w},b}(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f_{\vec{w},b}(\vec{x}^{(i)})) \right] \right\}$$

Interpretation:

- Low cost $\rightarrow$ Model predictions match actual labels well
- High cost $\rightarrow$ Model predictions don't match actual labels

Key Property: This cost function is convex for logistic regression, guaranteeing gradient descent finds the global minimum!
Name: This is also called "Binary Cross-Entropy Loss" or "Log Loss"

6 Gradient Descent for Logistic Regression

6.1 The Algorithm

Gradient Descent for Logistic Regression

Algorithm:

$$\begin{aligned} &\text{repeat until convergence}\{ \\ &\qquad w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b) \quad \text{for } j = 1, \ldots, n \\ &\qquad b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b) \\ &\qquad \} \end{aligned}$$

Gradient Formulas:

$$\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

$$\frac{\partial}{\partial b} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right)$$

where $f_{\vec{w},b}(\vec{x}^{(i)}) = \frac{1}{1+e^{-(\vec{w} \cdot \vec{x}^{(i)} + b)}}$
Important: All updates must happen simultaneously (use temporary variables)!

Gradient Descent: Logistic vs Linear Regression

Surprising Fact: The gradient formulas look IDENTICAL to linear regression!
Linear Regression:

$$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$$
$$\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

Logistic Regression:

$$f_{\vec{w},b}(\vec{x}) = \frac{1}{1 + e^{-(\vec{w} \cdot \vec{x} + b)}}$$
$$\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

Key Difference: The definition of $f_{\vec{w},b}(\vec{x})$ is different!

- Linear regression: f is linear
- Logistic regression: f includes sigmoid

Why the formulas look the same: The derivative of the log loss with respect to parameters has the same form as squared error, but the underlying model is different!

Complete Gradient Descent Algorithm

Initialize: Set $w_1, w_2, \ldots, w_n, b$ to small random values or zeros
Repeat until convergence:
Step 1: Compute predictions for all training examples

$$f_{\vec{w},b}(\vec{x}^{(i)}) = \frac{1}{1 + e^{-(\vec{w} \cdot \vec{x}^{(i)} + b)}} \quad \text{for } i = 1, \ldots, m$$

Step 2: Compute gradients

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} \quad \text{for } j = 1, \ldots, n$$
$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})$$

Step 3: Update parameters (simultaneously!)

$$w_j := w_j - \alpha \frac{\partial J}{\partial w_j} \quad \text{for } j = 1, \ldots, n$$
$$b := b - \alpha \frac{\partial J}{\partial b}$$

Step 4: Check convergence

- If cost J stops decreasing significantly, stop
- Or if maximum iterations reached, stop

Vectorized Implementation

Using NumPy for efficiency:

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def compute_gradient(X, y, w, b):
    m, n = X.shape

    # Predictions for all examples (vectorized)
    z = X @ w + b  # Matrix-vector multiplication
    f_wb = sigmoid(z)

    # Errors
    errors = f_wb - y

    # Gradients (vectorized)
    dj_dw = (1/m) * (X.T @ errors)  # Shape: (n,)
    dj_db = (1/m) * np.sum(errors)  # Scalar

    return dj_dw, dj_db

def gradient_descent(X, y, w_init, b_init, alpha, num_iters):
    w = w_init.copy()
    b = b_init

    for i in range(num_iters):
        # Compute gradients
        dj_dw, dj_db = compute_gradient(X, y, w, b)

        # Update parameters
        w = w - alpha * dj_dw
        b = b - alpha * dj_db

        # Optionally compute and print cost
        if i % 100 == 0:
            cost = compute_cost(X, y, w, b)
            print(f"Iteration {i}: Cost {cost}")

    return w, b
```

7 Overfitting in Classification

7.1 Understanding Overfitting

Three Scenarios in Classification

Consider binary classification with 2 features (x_1, x_2):

1. Underfitting (High Bias):

- **Model**: Too simple (e.g., linear boundary when data needs curve)
- **Symptoms**: High training error, high test error
- **Boundary**: Doesn't capture the pattern, misclassifies many points
- **Example**: Straight line trying to separate circular clusters

2. Good Fit (Just Right):

- **Model**: Appropriate complexity
- **Symptoms**: Low training error, low test error
- **Boundary**: Captures the true pattern, generalizes well
- **Example**: Smooth curve that separates classes cleanly

3. Overfitting (High Variance):

- **Model**: Too complex (e.g., high-degree polynomial)
- **Symptoms**: Very low training error, high test error
- **Boundary**: Wiggly, contorted, fits training data perfectly including noise
- **Example**: Complex curve with many twists trying to perfectly separate all points

Visual Understanding

Underfitting:

- Decision boundary is too simple
- Many training points on wrong side of boundary
- Model hasn't learned the pattern

Good Fit:

- Decision boundary separates classes well
- Most training points correctly classified
- Boundary is smooth and reasonable

Overfitting:

- Decision boundary is extremely wiggly
- Every training point perfectly classified
- Boundary has unrealistic curves around individual points
- Will perform poorly on new data

Key Problem with Overfitting: Model memorizes training data (including noise) instead of learning the underlying pattern!

When Does Overfitting Occur?

Common Causes:

1. **Too many features**
   - Many polynomial features: $x_1, x_2, x_1^2, x_2^2, x_1x_2, x_1^3, \ldots$
   - Model becomes too flexible
2. **Too few training examples**
   - Not enough data to constrain the model
   - Model fits noise in the small dataset
3. **Complex model for simple problem**
   - Using high-degree polynomial when linear would work
   - More parameters than necessary

Telltale Signs:

- Training accuracy: 98-100% (nearly perfect)
- Test accuracy: 60-70% (poor)
- Large gap between training and test performance
- Decision boundary looks unreasonably complex

8 Addressing Overfitting

8.1 Three Main Strategies

Strategy 1: Collect More Training Data

How it helps:

- More data points constrain the model
- Model can't memorize all examples
- Forces model to learn general patterns
- Noise averages out with more data

When to use:

- You have access to more data
- Cost of collecting data is reasonable
- Current dataset is small relative to problem complexity

Limitations:

- Not always possible (data is expensive or unavailable)
- Doesn't address fundamental model complexity issues

Example: For house price prediction:

- 10 training examples + 20 features $\rightarrow$ likely overfits
- 10,000 training examples + 20 features $\rightarrow$ can generalize well

Strategy 2: Feature Selection

Goal: Reduce number of features to only the most important ones
Methods:

- **Manual selection**:
  - Use domain knowledge
  - Keep features that make intuitive sense
  - Remove redundant or irrelevant features
- **Automatic selection**:
  - Forward selection: Start with no features, add best one at a time
  - Backward elimination: Start with all features, remove worst one at a time
  - Feature importance ranking

Example: For house prices with many features:

- Keep: size, bedrooms, location, age (clearly relevant)
- Remove: distance to coffee shop, neighbor's age (likely irrelevant)

Trade-off:

- **Benefit**: Reduces overfitting, simpler model
- **Risk**: Might throw away useful information

Strategy 3: Regularization

Idea: Keep all features but reduce their impact by penalizing large parameter values
How it works:

- Don't eliminate features completely
- Instead, shrink their weights toward zero
- Features with little predictive power get weights close to zero
- Important features keep larger weights

Benefits:

- Keeps all features (doesn't throw away information)
- Automatically determines feature importance
- Works well when many features have small effects

When to use:

- You have many features but don't know which are important
- Features might all contribute a little
- You want automatic feature importance weighting

Preview: Regularization is the most commonly used approach in practice!

9 Regularization

9.1 The Concept

What is Regularization?

Core Idea: Add a penalty to the cost function for having large parameter values
Intuition:

- Large weights → complex model → more prone to overfitting
- Small weights → simpler model → better generalization
- Regularization encourages small weights

Analogy: Like a budget constraint

- Without regularization: Model can use any weights (unlimited budget)
- With regularization: Model must "pay" for large weights (limited budget)
- Model must "spend wisely" on important features

Plain English: Regularization tells the model "You can keep all the features, but you'll be penalized for using large weights. Choose wisely which features to emphasize!"

9.2 Regularization Term

Regularization Term

Formula:

$$\text{Regularization term} = \frac{\lambda}{2m} \sum_{j=0}^{n-1} w_j^2$$

or sometimes written as:

$$\text{Regularization term} = \frac{\lambda}{2m} \sum_{j=1}^{n} w_j^2$$

Components:

- λ (lambda): **Regularization parameter**
  - Controls strength of regularization
  - $\lambda = 0$: No regularization (might overfit)
  - λ very large: Strong regularization (might underfit)
  - Typical values: $0.01, 0.1, 1, 10, 100$
- w_j^2: Square of each weight
  - Penalizes large positive and negative weights equally
  - Quadratic penalty grows quickly with weight magnitude
- $\sum_{j=0}^{n-1} w_j^2$: Sum of squared weights
  - Also called L^2 norm squared: $\|\vec{w}\|^2$
  - Measures total "size" of weight vector

Important: We typically do NOT regularize the bias term b

- Only regularize weights w_j
- Bias b just shifts the model, doesn't affect complexity

Understanding Lambda (λ)

Effect of Different λ Values:
$\lambda = 0$ **(No Regularization)**:

- Cost function = prediction error only
- Model can use any weight values
- Risk of overfitting if too many features

λ **Small (Light Regularization)**:

- Slight penalty for large weights
- Model still fairly flexible
- Good when you have reasonable amount of data

λ **Medium (Moderate Regularization)**:

- Balanced penalty for large weights
- Model complexity is constrained
- Often the best choice in practice

λ **Very Large (Strong Regularization)**:

- Heavy penalty for large weights
- Weights forced to be very small (close to 0)
- Model becomes too simple $\rightarrow$ underfitting
- Decision boundary becomes nearly flat/linear

Choosing λ: Use cross-validation to find optimal value!

9.3 Regularized Cost Function

Regularized Logistic Regression Cost

Cost Function with Regularization:

$$J(\vec{w}, b) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log(f_{\vec{w},b}(\vec{x}^{(i)})) + (1-y^{(i)})\log(1-f_{\vec{w},b}(\vec{x}^{(i)}))\right] + \frac{\lambda}{2m}\sum_{j=0}^{n-1} w_j^2$$

Two parts:

1. **Loss term** (first part): Measures prediction error
   - How well model fits training data
   - Want this to be small
2. **Regularization term** (second part): Penalizes large weights
   - Measures model complexity
   - Want this to be small too

Trade-off:

- Minimize loss → fit training data well (but risk overfitting)
- Minimize regularization → keep weights small (but risk underfitting)
- λ controls the balance between these two goals

How Regularization Prevents Overfitting

Without Regularization:

$$\min_{\vec{w},b} J(\vec{w}, b) = \min_{\vec{w},b}\{\text{prediction error}\}$$

- Model only cares about fitting training data
- Can use arbitrarily large weights
- Leads to overfitting

With Regularization:

$$\min_{\vec{w},b} J(\vec{w}, b) = \min_{\vec{w},b}\{\text{prediction error} + \lambda \times \text{weight size}\}$$

- Model must balance two goals:
  1. Fit training data (low prediction error)
  2. Keep weights small (low regularization term)
- Can't fit training data perfectly if it requires large weights
- Forced to find simpler, more generalizable solution

Result: More robust model that performs better on new data!

10 Regularized Gradient Descent

10.1 Updated Gradient Formulas

Gradients with Regularization

Cost Function:

$$J(\vec{w}, b) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log(f_{\vec{w},b}(\vec{x}^{(i)})) + (1-y^{(i)})\log(1-f_{\vec{w},b}(\vec{x}^{(i)}))\right] + \frac{\lambda}{2m}\sum_{j=0}^{n-1} w_j^2$$

Gradient for weights:

$$\frac{\partial}{\partial w_j}J(\vec{w}, b) = \frac{1}{m}\sum_{i=1}^{m}\left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}\right)x_j^{(i)} + \frac{\lambda}{m}w_j$$

Gradient for bias (unchanged!):

$$\frac{\partial}{\partial b}J(\vec{w}, b) = \frac{1}{m}\sum_{i=1}^{m}\left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}\right)$$

Key Difference:

- Weight gradient has extra term: $\frac{\lambda}{m}w_j$
- Bias gradient stays the same (no regularization for b)

Understanding the Regularization Gradient

Gradient for w_j has two parts:

$$\frac{\partial}{\partial w_j}J(\vec{w}, b) = \underbrace{\frac{1}{m}\sum_{i=1}^{m}(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})x_j^{(i)}}_{\text{Prediction error gradient}} + \underbrace{\frac{\lambda}{m}w_j}_{\text{Regularization gradient}}$$

First part (prediction error gradient):

- Same as without regularization
- Points in direction to reduce prediction error
- Wants to fit training data better

Second part (regularization gradient):

- New term from regularization
- Proportional to current weight value w_j
- Always points toward zero (shrinks weights)
- Strength controlled by λ

Combined effect: Gradient balances fitting data and keeping weights small!

10.2 Gradient Descent Algorithm

Regularized Gradient Descent

Algorithm:

repeat until convergence{

$$w_j = w_j - \alpha \left[\frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} w_j \right] \quad \text{for } j = 0, \ldots, n-1$$

$$b = b - \alpha \left[\frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) \right]$$

}

Alternative form for weights:

$$w_j = w_j - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} - \alpha \frac{\lambda}{m} w_j$$

Rearranging:

$$w_j = w_j \left(1 - \alpha \frac{\lambda}{m}\right) - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

Interpretation: Each iteration:

1. **Shrink** w_j by factor $(1 - \alpha \frac{\lambda}{m})$ (usually ≈ 0.99)
2. **Update** based on prediction error

This is sometimes called "weight decay"!

Vectorized Implementation with Regularization

NumPy implementation:

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def compute_cost_reg(X, y, w, b, lambda_):
    m = X.shape[0]

    # Predictions
    z = X @ w + b
    f_wb = sigmoid(z)

    # Loss term (binary cross-entropy)
    loss = -y * np.log(f_wb) - (1 - y) * np.log(1 - f_wb)
    cost_without_reg = (1/m) * np.sum(loss)

    # Regularization term
    reg_cost = (lambda_/(2*m)) * np.sum(w**2)

    total_cost = cost_without_reg + reg_cost
    return total_cost

def compute_gradient_reg(X, y, w, b, lambda_):
    m, n = X.shape

    # Predictions
    z = X @ w + b
    f_wb = sigmoid(z)

    # Errors
    errors = f_wb - y

    # Gradients with regularization
    dj_dw = (1/m) * (X.T @ errors) + (lambda_/m) * w
    dj_db = (1/m) * np.sum(errors)

    return dj_dw, dj_db

def gradient_descent_reg(X, y, w_init, b_init, alpha, lambda_, num_iters):
    w = w_init.copy()
    b = b_init

    for i in range(num_iters):
        # Compute gradients
        dj_dw, dj_db = compute_gradient_reg(X, y, w, b, lambda_)

        # Update parameters
        w = w - alpha * dj_dw
        b = b - alpha * dj_db

        # Optionally print cost
        if i % 100 == 0:
            cost = compute_cost_reg(X, y, w, b, lambda_)
            print(f"Iteration {i}: Cost {cost:.4f}")
```

Gradient Descent: With vs Without Regularization

Without Regularization:

$$w_j = w_j - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$
$$b = b - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})$$

With Regularization:

$$w_j = w_j - \alpha \left[\frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} w_j \right]$$
$$b = b - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})$$

Summary of Changes:

- **Weights**: Add $\frac{\lambda}{m} w_j$ to gradient
- **Bias**: No change (not regularized)
- **Effect**: Weights are pulled toward zero each iteration

11 Key Formulas Summary

Everything You Need to Memorize

1. Sigmoid Function:

$$g(z) = \frac{1}{1 + e^{-z}}$$

2. Logistic Regression Model:

$$f_{\vec{w},b}(\vec{x}) = g(\vec{w} \cdot \vec{x} + b) = \frac{1}{1 + e^{-(\vec{w} \cdot \vec{x} + b)}}$$

3. Decision Boundary:

$$\vec{w} \cdot \vec{x} + b = 0$$

Predict $\hat{y} = 1$ if $\vec{w} \cdot \vec{x} + b \geq 0$, else $\hat{y} = 0$

4. Cost Function (Binary Cross-Entropy):

$$J(\vec{w}, b) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log(f_{\vec{w},b}(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f_{\vec{w},b}(\vec{x}^{(i)})) \right]$$

5. Regularized Cost Function:

$$J(\vec{w}, b) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log(f_{\vec{w},b}(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f_{\vec{w},b}(\vec{x}^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=0}^{n-1} w_j^2$$

6. Gradient Descent (Without Regularization):

$$w_j = w_j - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})$$

7. Gradient Descent (With Regularization):

$$w_j = w_j - \alpha \left[\frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} w_j \right]$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})$$

12 Summary and Key Concepts

Essential Points

1. **Classification vs Regression**:
   - Regression: Predict continuous values
   - Classification: Predict discrete categories (binary: 0 or 1)
2. **Sigmoid Function**:
   - Formula: $g(z) = \frac{1}{1+e^{-z}}$
   - Maps any real number to $(0, 1)$
   - Outputs represent probabilities
   - S-shaped curve with midpoint at 0.5
3. **Logistic Regression Model**:
   - $f_{\vec{w},b}(\vec{x}) = g(\vec{w} \cdot \vec{x} + b)$
   - Linear combination + sigmoid = probability
   - Output: $P(y = 1|\vec{x})$
   - Threshold at 0.5 for binary decision
4. **Decision Boundaries**:
   - Linear: Straight line/hyperplane where $\vec{w} \cdot \vec{x} + b = 0$
   - Non-linear: Use polynomial features for curves
   - Separates positive class from negative class
5. **Cost Function**:
   - Binary cross-entropy (log loss)
   - Penalizes confident wrong predictions heavily
   - Convex $\rightarrow$ gradient descent finds global minimum
   - Different from squared error used in regression
6. **Gradient Descent**:
   - Update formulas look same as linear regression
   - But $f_{\vec{w},b}(\vec{x})$ includes sigmoid
   - Simultaneous updates of all parameters
   - Iteratively minimizes cost function
7. **Overfitting**:
   - Model too complex $\rightarrow$ fits training data perfectly but fails on new data
   - Symptoms: Low training error, high test error
   - Causes: Too many features, too few examples
   - Decision boundary becomes unreasonably wiggly
8. **Addressing Overfitting**:

Comparison: Linear vs Logistic Regression

Aspect	Linear Regression	Logistic Regression
Task	Regression	Classification
Output	Continuous value	Probability (0 to 1)
Model	$\vec{w} \cdot \vec{x} + b$	$g(\vec{w} \cdot \vec{x} + b)$
Range	$(-\infty, \infty)$	$(0, 1)$
Cost	Squared error	Binary cross-entropy
Interpretation	Predicted value	$P(y = 1\|\vec{x})$
Decision	Direct prediction	Threshold at 0.5
Similarities		
Parameters	Weights $\vec{w}$ and bias b	
Optimization	Gradient descent	
Gradient form	Similar structure (different f)	
Regularization	Both use L^2 penalty	

Looking Ahead

What's Next?

- **Multi-class Classification**: Extending to more than 2 classes
- **Neural Networks**: Stacking logistic units for complex patterns
- **Support Vector Machines**: Alternative classification algorithm
- **Decision Trees**: Non-linear classification without feature engineering
- **Evaluation Metrics**: Precision, recall, F1-score, ROC curves

The Foundation: Logistic regression is the building block for:

- Neural networks (logistic units as neurons)
- Deep learning (many layers of logistic units)
- Modern machine learning models

Master logistic regression first - it's fundamental to everything else!

Practical Tips

When using Logistic Regression:

1. **Feature scaling**: Still important! Use z-score normalization
2. **Learning rate**: Start with $\alpha = 0.01$ or 0.1, adjust as needed
3. **Regularization**: Try $\lambda \in \{0.01, 0.1, 1, 10, 100\}$
4. **Convergence**: Monitor cost function, should decrease each iteration
5. **Evaluation**: Use training AND test sets to detect overfitting
6. **Polynomial features**: Add carefully, watch for overfitting
7. **Class imbalance**: If classes very imbalanced, use special techniques

Common Mistakes to Avoid:

- Using linear regression for classification (wrong!)
- Forgetting to apply sigmoid function
- Not checking for overfitting (only looking at training accuracy)
- Setting λ too large (causes underfitting)
- Not scaling features (slows convergence)